HealthWithSevgi

A Browser-Based ML Education Tool for Healthcare Professionals

Final Project Report — 2-Page Summary

EudaLabs Team

Efe Çelik	Burak Aydoğmuş	Batuhan Bayazıt	Berat Mert Gökkaya	Berfin Duru Alkan
202128016	202128028	202228008	202228019	202228005
PO + Dev	*UX*	*Lead Dev + SM*	*Dev*	*QA Lead*

SENG 430 — Software Quality Assurance

Çankaya University · Spring 2025–2026

5 May 2026

HealthWithSevgi guides healthcare professionals through a seven-step machine-learning pipeline — from clinical context to ethics audit — across 20 medical specialties, with no programming required. It ships as a single-container HuggingFace Space pre-built and pulled from the GitHub Container Registry.

1. Architecture

The system is a two-tier web application packaged as a single deployable container. The browser receives a React 18 + Vite + TypeScript single-page application; all stateful operations are delegated to a FastAPI backend over HTTPS. Backend services are singletons attached to `app.state` and consumed in routers via `request.app.state.<service>`, organised into three thin route files (`data_router`, `ml_router`, `explain_router`). All session data — uploaded CSVs, trained models, SHAP values — lives in a per-process LRU cache; nothing touches a database. The ML engine wraps `scikit-learn 1.4+`, `SHAP 0.45+`, and `XGBoost / LightGBM` across eight classifier families. Clinical insights at Step 7 are produced by Google's **Gemini 2.5 Flash** via the AI Studio REST API (with an optional MedGemma fall-through on Vertex AI when configured), and a deterministic template as final fallback when the LLM is unavailable. In production, `hf-space/main_hf.py` serves both the API and the static Vite build from a single uvicorn process on port 7860; locally, a Vite proxy forwards `/api` to a separate uvicorn process. Distribution is via GHCR (`ghcr.io/eudalabs/healthwithsevgi:latest`) with a `docker-compose.yml` that pulls the pre-built image and falls back to a local multi-stage build.

2. Key Decisions

- **No database.** All clinical data stays in-memory and is evicted as the LRU fills. Eliminates a class of privacy risks and removes a dependency for graders running clean-room.
- **Single-container deployment.** The same image runs locally via `docker compose up`, on HuggingFace Spaces, and on a clean grader machine. `main_hf.py` mounts the Vite build under uvicorn so there is no second process to supervise.
- **Gemini 2.5 Flash with thinking-aware parsing.** A brief migration to Gemma 4 (v1.5.8) was reverted in favour of Gemini 2.5 Flash for stability. The response parser still drops parts marked `thought=true` so any chain-of-thought tokens never leak into the clinical assessment card.
- **Lazy-loaded wizard steps.** Steps 2 through 7 are dynamic-imported via `React.lazy`. Recharts is no longer eagerly preloaded at landing — landing JS dropped by 70 KiB and Lighthouse SEO climbed from 91 to 100.
- **Accessibility without opacity.** Locked wizard steps formerly used `opacity: 0.45` to indicate disabled state; opacity stacked with the parent surface and broke the WCAG 1.4.3 contrast target. Replaced with `var(-text-secondary)` which clears AA at 5.85:1.
- **GHCR-first compose.** `docker-compose.yml` reaches for the pre-built image first; the local build context is a fallback. Graders need only `docker compose up` — no source checkout.

3. Challenges

Step-7 LLM reliability. The original 45 s axios + 45 s server timeout was too tight for the Step-7 reasoning calls: long ethics prompts occasionally crossed 60 s and the frontend rendered empty cards on every other request. Resolution: backend HTTP timeout raised to 200 s, one jittered retry on `ReadTimeout / 429 / 5xx`, axios bumped to 450 s, and a graceful *"assessment unavailable, reload to retry"* alert when the template fallback fires instead of a blank card. Verified across five consecutive multi-task calls (15 / 15 successful Gemini responses).

Sprint-4 retrospective debt. Five bugs surfaced in the Sprint-4 retro (`SCRUM-217..221`, 11 pts) — a missing `ErrorBoundary` around lazy steps, silent error suppression in Step 6, a parallel-coordinates crash on null metric, and two related null-handling regressions. They were re-prioritised into Sprint 5 and *shipped in the first half* of the sprint window so polish and user-testing landed on a stable trunk.

Cross-browser regression caught late. UT-03 caught a Safari-specific viewport regression in the parallel-coordinates chart only at audit time. The fix was a one-line `min-height`; the discovery cost a half-day. The standing recommendation is a Playwright matrix in CI running Chrome / Firefox / Safari / Edge on every PR.

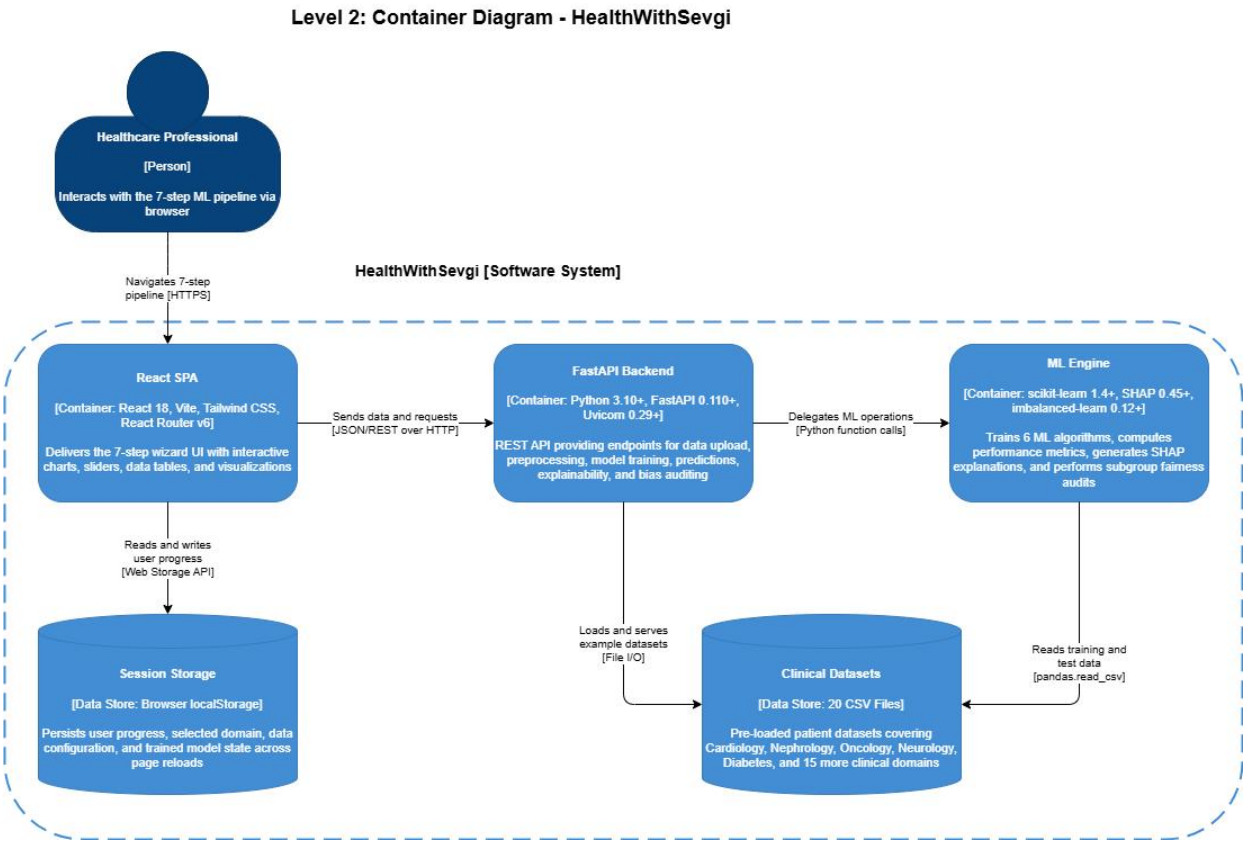

Figure 1: C4 Level-2 Container view of HealthWithSevgi. Source under `docs/drawio/`.

20 specialties × 7 steps × 3 CSV variants. Building enough end-to-end coverage for a five-person team to credibly claim *"no crashes across all clinical specialties"* required automation: 140 cases for full-domain coverage and 21 cases for the 3-CSV regression suite. Together they document 161 / 161 passing cases with zero failures (`Sprint5_Full_Domain_Coverage.pdf`, `Sprint5_E2E_Regression.pdf`).

4. Lessons Learned

- **Book user-testing windows at the *start* of the sprint, not the end.** Recording and signed consent slipped to the final 48 hours because both were scheduled after polish. The fix is administrative: lock the participant slot during sprint planning.
- **One non-CS user is the floor, not the bar.** P1 returned 7 / 7 task pass with SUS 90, but a single observation cannot triangulate a population. Two follow-up sessions (clinician + administrator) are queued for any post-academic continuation.
- **Quality scores need a CI floor.** Lighthouse Perf 91 / A11y 100 / BP 100 / SEO 100 was achieved manually. Without a GitHub Action gating `main` on a Lighthouse threshold, any future contributor can silently regress jury-grade scores. The proposed gate is Perf $\geq$ 85 and A11y $\geq$ 95, evaluated against the production HF build.
- **Documentation is a deliverable, not a chore.** Frontend JSDoc and backend `interrogate` both reached 100 % in Sprint 5. Instituting the rule from Sprint 1 would have spread the cost across five sprints rather than concentrating it in the last one.
- **Reasoning models need different timeouts than chat models.** The Step-7 incident is the durable lesson: any LLM with chain-of-thought reasoning needs an SLA per call that accounts for the reasoning pass, plus parser-level filtering of `thought` parts before they reach the user.

Live demo: https://0xbatuhan4-healthwithsevgi.hf.space/ *Source:* https://github.com/EudaLabs/HealthWithSevgi *Wiki:* https://github.com/EudaLabs/HealthWithSevgi/wiki *Container:* https://github.com/EudaLabs/HealthWithSevgi/pkgs/container/healthwithsevgi